Modelado De Datos Con Estructuras (STRUCT) Y Punteros

Luca Di Guglielmo Villa
E.E.S.T N°1 de Vicente López
Curso: Laboratorio de Programación 5°3°
Profesor: Ganduglia Yamil & York Elias Mansilla Muñoz
Fecha: 14/10/2026

Introducción

En esta actividad, se busca el aprendizaje sobre que es una estructura (struct), los operadores flecha y otros conceptos vinculados a la Heterogeneidad, Eficiencia en RAM, el Buffer de Entrada y la posible aplicación sobre el proyecto integrador

Desarrollo

Para empezar esta actividad, cree la estructura del repositorio de forma local en mi PC, una vez hecho esto, pegue el código base proporcionado y lo ejecute en la terminal.

Programa en ejecución

src > main.cpp > main()

```cpp
struct EntidadProyecto {
    float metrica; // Representa lecturas de sensores, consumo, avance, etc.
};

// Prototipo de función con pasaje por dirección mediante puntero
void cargarDatos(EntidadProyecto* ptr);

int main() {
    // Inicialización de seguridad en la memoria Stack
    EntidadProyecto miEntidad = {0, "Vacio - Luca Di Guglielmo", 0.0f};

    cout << "======================================================" << endl;
    cout << "  MODELADO STRUCT - ESTUDIANTE: Luca Di Guglielmo" << endl;
    cout << "======================================================" << endl;

    // Invocación enviando la dirección física de memoria con '&'
    cargarDatos(&miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
    cout << "ID Registrado:     " << miEntidad.id << endl;
    cout << "Nombre Registrado: " << miEntidad.nombre << endl;
    cout << "Metrica Guardada:  " << miEntidad.metrica << endl;
    cout << "Direccion RAM Hexadecimal: " << &miEntidad << endl;
    cout << "======================================================" << endl;

    return 0; // Código 0: Finalización exitosa
}

void cargarDatos(EntidadProyecto* ptr) {
    cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;

    cout << "=> Ingrese el ID de la entidad (entero): ";
    cin >> ptr->id;

    // Limpieza del buffer obligatoria antes de cin.getline
  cin.ignore();

    cout << "=> Ingrese el Nombre o Descripcion: ";
    cin.getline(ptr->nombre, 50);

    cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";
    cin >> ptr->metrica;
}
```

Problems Output Debug Console **Terminal** Ports

```
Nombre Registrado:
Metrica Guardada:  0
Direccion RAM Hexadecimal: 0xa9fe9ffc00
======================================================
PS C:\Users\lucad\Documents\actividad5>
PS C:\Users\lucad\Documents\actividad5>
PS C:\Users\lucad\Documents\actividad5>
PS C:\Users\lucad\Documents\actividad5> .\src\main.exe
======================================================
  MODELADO STRUCT - ESTUDIANTE: [Su Nombre y Apellido]
======================================================

--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---
=> Ingrese el ID de la entidad (entero): 10
=> Ingrese el Nombre o Descripcion: nombre
=> Ingrese la Metrica de Operacion (decimal/float): 3003.53

=== DATOS VERIFICADOS EN LA MEMORIA RAM ===
ID Registrado:     10
Nombre Registrado: nombre
Metrica Guardada:  3003.53
Direccion RAM Hexadecimal: 0x6ae7dffd50
======================================================
PS C:\Users\lucad\Documents\actividad5>
```

Actividades:

1. Agrupar los datos de un sensor en un struct es mejor que agruparlas sueltas porque: La organización lógica del código es mejor ya que se guardan los datos en una misma entidad (el struct); es más manejable, en una funcion solo tienes que enviar un parametro con toda la estructura en vez de muchas variables por separado; la escalabilidad es mucho más simple gracias a que una nueva variable se guarda junto a todos los otras en un mismo lugar; y por ultimo, la coherencia que se garantiza gracias a que estas variables 'viajan' juntas.

2. Esto no se puede realizar porque el operador punto solo sirve para miembros de un objeto de forma directa, osea que el valor que se une con el punto no debe ser un puntero, para utilizarlo primero se debe desreferenciar el dato (para esto sirve el operador flecha)

3. El pasaje por referencia es más eficiente ya que el puntero al ser pasado, por ejemplo, en una función, simplemente envia la direccion donde se almacena el struct, lo que hace innecesario la copia del struct en una nueva ubicación de RAM

4. cin.ignore() elimina el \n (indicador de nueva línea) de una función, al querer ingresar un dato mediante cin y otro con getline() (como el caso del ingreso de edad y nombre), ocurre que, al intentar ingresar el nombre, la función getline() toma un valor vacío, ya que esta lee hasta el indicador de nueva línea; por eso es importante la adición de cin.ignore

5. Un ejemplo que se puede aplicar en el proyecto de GAMY (proyecto en el cual soy miembro), es guardar los datos del jugador, estos serían:

- int monedasRecogidas
- int cantidadMuertes
- float monedasPorMuertes

Conclusión

Durante la realización de la actividad se trabajó sobre conceptos importantes como las estructuras, el ahorro de uso de la ram mediante punteros, los operadores flecha y como aplicar esto sobre un código funcional

Referencias:

Luca Di Guglielmo
Github: https://github.com/specIux/actividad-5-LPR.git

Laboratorio de Programación 5°3°
Actividad: 💬📝📺🛠️ ACTIVIDAD 5: MODELADO DE DATOS CON ESTRUCTURAS (STRU…